

React Native Core Components: 100 Master Interview Questions & Answers

This document is formatted as a structured reference guide (ready to copy directly into Microsoft Word or Google Docs). It covers essential technical interview questions focused exclusively on React Native's primary core UI components: `View`, `Text`, `Image`, `Button` / `Pressable`, `TextInput`, `ScrollView`, `FlatList`, and `SafeAreaView` / `StatusBar`.

Section 1: The `View` Component (Questions 1–12)

Q1: What is the React Native `view` component, and what is its Web DOM equivalent?

- **Answer:** `View` is the fundamental building block for UI in React Native, equivalent to a `<div>` on the web. It is a non-scrolling container that maps directly to native views (`UIView` on iOS, `android.view.View` on Android) and supports Flexbox layout.

Q2: Can you render raw text directly inside a `view` without a `Text` tag?

- **Answer:** No. Placing raw text directly inside a `<View>` causes a runtime error (e.g., *"Text strings must be rendered within a <Text> component"*).

Q3: How does layout direction in React Native `view` differ from web Flexbox?

- **Answer:** In standard web CSS, `flexDirection` defaults to `row`. In React Native `View`, `flexDirection` defaults to `column`.

Q4: Does `view` support traditional CSS grid or absolute float positioning?

- **Answer:** React Native `view` does not support CSS Grid or floating (`float`). It relies entirely on Flexbox and absolute positioning (`position: 'absolute'`).

Q5: What is the difference between `pointerEvents` props on a `view`?

- **Answer:** `pointerEvents` controls whether a `View` handles touch events. Options include:
 - `'auto'`: Normal touch target behavior.
 - `'none'`: Ignores touches and lets them pass through to underlying views.
 - `'box-none'`: Ignores touches on the container itself but allows touches on its children.
 - `'box-only'`: Handles touches on the container but ignores children.

Q6: How does `View` handle clipping of off-screen or overflowing child components?

- **Answer:** By default, child content that exceeds `View` boundaries may render outside unless `overflow: 'hidden'` is applied.

Q7: What prop allows you to measure the exact dimensions and position of a `View` on layout?

- **Answer:** The `onLayout` prop, which receives an event payload containing `{ x, y, width, height }` relative to the parent.

Q8: How do background colors and border radii behave differently on iOS vs. Android `Views`?

- **Answer:** On iOS, setting `borderRadius` on a `View` with overflow content requires `overflow: 'hidden'` to clip children. On Android, `borderRadius` clips background rendering natively, but complex subviews may behave differently without explicit elevation.

Q9: What is the purpose of the `collapsible` prop on Android `view` components?

- **Answer:** It is an Android-specific optimization. Setting `collapsible={true}` (default) allows the native layout engine to merge views that only exist for layout styling, reducing native view hierarchy depth.

Q10: How do you implement shadows on a `view` cross-platform?

- **Answer:** iOS uses Shadow Props (`shadowColor`, `shadowOffset`, `shadowOpacity`, `shadowRadius`). Android relies on the `elevation` prop. Libraries like `react-native-shadow-2` or SVG helpers are often used for uniform cross-platform shadows.

Q11: What is the difference between `margin` and `padding` behavior in a React Native `view`?

- **Answer:** `margin` controls space outside the `View` boundaries, whereas `padding` creates space inside the `View` boundaries between the frame and its children.

Q12: Why shouldn't you wrap large list elements inside generic `views` with fixed heights?

- **Answer:** Generic `View` components are not virtualized; mounting large sequences of `View` nodes exhausts native GPU/RAM memory quickly.

Section 2: The `Text` Component (Questions 13–24)

Q13: Why is the `Text` component unique compared to web typography handling?

- **Answer:** `Text` is the *only* component allowed to host text nodes. It handles native typography pipelines (`UILabel` / `TextView`) and automatically inherits style from nested `Text` elements (unlike sibling or parent `View` elements).

Q14: Does style inheritance work from `View` to `Text` in React Native?

- **Answer:** No. Styles applied to a `<View>` (like `fontSize` or `color`) do *not* inherit down to `<Text>`. Style inheritance only works between nested `<Text>` elements (e.g., `<Text style="{a}"><Text style="{b}">...</Text></Text>`).

Q15: How do you truncate long text with an ellipsis in React Native?

- **Answer:** Set `numberOfLines={1}` (or desired limit) and `ellipsizeMode="tail"` (or `"head"`, `"middle"`, `"clip"`).

Q16: What happens if you wrap plain text inside conditional logic like `{condition && 'Hello'}` when `condition` is 0?

- **Answer:** React Native attempts to render 0 as a raw string. Since 0 is outside a `<Text>` node, it throws a runtime crash. Fix: `{Boolean(condition) && <Text>Hello</Text>}`.

Q17: What is the `allowFontScaling` prop used for in `Text`?

- **Answer:** It controls whether the text size scales according to device accessibility settings (OS-level font size preferences). Setting it to `false` locks the font size.

Q18: How do custom fonts work in React Native `<Text>`?

- **Answer:** Custom fonts are linked as assets (`ttf/otf`) and applied via `fontFamily: 'FontName'`. The font name must match the exact embedded postscript font name (iOS) or filename (Android).

Q19: What prop handles user selection and copying of text content?

- **Answer:** The `selectable={true}` prop allows users to select, highlight, and copy text to their clipboard.

Q20: How do line heights (`lineHeight`) behave differently between iOS and Android in `<Text>`?

- **Answer:** On iOS, `lineHeight` centers text vertically within the line frame. On Android, `lineHeight` expands space above the text baseline, which can shift visual alignment if not tested on both platforms.

Q21: What is the difference between `onPress` on a `<View>` VS `onPress` on a `<Text>`?

- **Answer:** `<Text onPress="{...}">` natively handles touch inline within paragraph boundaries without wrapping the string inside a separate button frame.

Q22: What is `maxFontSizeMultiplier`?

- **Answer:** A prop on `<Text>` that caps the maximum scale factor allowed when accessibility font scaling (`allowFontScaling`) is enabled, preventing layout breakages.

Q23: How do inline images work inside a `<Text>` component?

- **Answer:** You can place an `<Image>` component as a direct child inside a `<Text>` tree. The system renders the image inline with surrounding text wrapping.

Q24: What is the performance impact of deeply nested `<Text>` components?

- **Answer:** Heavy nesting increases text measurement calculation cost during the layout pass (Yoga engine), which can cause frame drops during fast re-renders.

Section 3: The `Image` Component (Questions 25–38)

Q25: What are the two primary image sources in React Native, and how are they declared?

- **Answer:**
 1. **Local Images:** Declared via `require('./path/image.png')` (dimensions are auto-detected at build/compile time).
 2. **Remote Images:** Declared via `{ uri: '[https://example.com/image.png] (https://example.com/image.png) '` (requires explicit width and height).

Q26: Why does a remote image fail to display if `width` and `height` are not set?

- **Answer:** Remote images are loaded asynchronously. Without explicit dimensions, the `Image` component defaults to `$0 \times 0$` pixels.

Q27: List all valid values for the `resizeMode` prop on `Image`.

- **Answer:** `'cover'`, `'contain'`, `'stretch'`, `'repeat'`, and `'center'`.

Q28: How does `Image.prefetch()` work?

- **Answer:** `Image.prefetch(url)` downloads a remote image to local disk cache ahead of time, ensuring instant rendering when requested by UI.

Q29: What is the difference between `Image` and `ImageBackground`?

- **Answer:** `Image` is a standalone leaf component that cannot take children. `ImageBackground` is a `<View>` wrapper that accepts children overlaid on top of a background image.

Q30: How do you handle fallback images when a remote image fails to load?

- **Answer:** Use the `onError` prop callback on `<Image>` to update state and toggle a local fallback image source.

Q31: What is the `defaultSource` prop in React Native `Image`?

- **Answer:** An iOS-specific prop that defines a local static placeholder image to display while the remote network image is downloading.

Q32: What prop on `Image` tracks downloading progress for large remote files?

- **Answer:** The `onProgress` prop returns a native event containing `{ nativeEvent: { loaded, total } }` (supported primarily on iOS).

Q33: How does React Native handle high-density screen resolutions (e.g., @2x, @3x) for local assets?

- **Answer:** React Native automatically selects `image@2x.png` or `image@3x.png` based on the target device's pixel density (`PixelRatio`).

Q34: What is progressive image loading, and how do you achieve it?

- **Answer:** Loading a low-resolution thumbnail or blurhash first, then transitioning smoothly (via opacity fade animation) to the high-resolution image using `onLoadEnd`.

Q35: How do you pass custom request headers (like JWT Bearer tokens) to fetch secured images?

- **Answer:** Pass custom headers in the image source object: `source={{ uri: '...', headers: { Authorization: 'Bearer token' } }}`.

Q36: Why can aggressive use of large, unoptimized images cause OOM (Out Of Memory) crashes?

- **Answer:** Decoded bitmaps in memory consume bytes proportional to image dimensions ($\text{\$Width} \times \text{\$Height} \times 4 \text{ bytes}$), not raw file size on disk.

Q37: What is `capInsets` used for on the `Image` component?

- **Answer:** It defines 9-patch style stretchable image borders (useful for resizable background bubble cards on iOS).

Q38: Why is the third-party library `react-native-fast-image` widely used instead of core `Image`?

- **Answer:** It provides aggressive disk/memory caching, priority loading, GIF support, and consistent cross-platform image performance backed by `SDWebImage` (iOS) and `Glide` (Android).

Section 4: `Button` VS. `Pressable` / `TouchableOpacity` (Questions 39–50)

Q39: Why is the core `<Button>` component rarely used in production apps?

- **Answer:** `<Button>` is a rigid, platform-native wrapper with minimal styling capability. Its appearance (blue text on iOS, material button on Android) cannot be easily customized with custom borders, icons, or complex layouts.

Q40: What are the primary touch-handling components available in React Native?

- **Answer:** `TouchableOpacity`, `TouchableHighlight`, `TouchableWithoutFeedback`, `TouchableNativeFeedback` (Android only), and `Pressable`.

Q41: Why is `Pressable` preferred over older `Touchable` components in modern React Native?

- **Answer:** `Pressable` is the newer, lower-level core primitive. It offers a cleaner functional API, handles state-based press feedback (`pressed`), supports hit slops, and incurs lower overhead than older `Touchable` implementations.

Q42: How does `hitSlop` prop work on `Pressable` / `TouchableOpacity`?

- **Answer:** `hitSlop` extends the touchable area beyond the component's visual bounds without altering layout measurements (e.g., `hitSlop={{ top: 10, bottom: 10, left: 10, right: 10 }}`).

Q43: How do you style a `Pressable` based on its interaction state?

- **Answer:** By passing a function to the `style` prop:

JavaScript

```
<Pressable style={({ pressed }) => [{ opacity: pressed ? 0.5 : 1.0 }]}>
```

Q44: What is the difference between `onPress`, `onLongPress`, `onPressIn`, and `onPressOut`?

- **Answer:**
 - `onPressIn`: Triggered immediately when finger touches down.
 - `onPressOut`: Triggered when finger lifts up or moves off.
 - `onPress`: Triggered upon tap completion.
 - `onLongPress`: Triggered when touch duration exceeds a threshold (typically ~500ms).

Q45: What prop on `TouchableOpacity` controls its active opacity during a tap?

- **Answer:** `activeOpacity` (defaults to 0.2).

Q46: How do you render Android ripple effects using core primitives?

- **Answer:** Use `Pressable` with the `android_ripple` prop: `android_ripple={{ color: '#ccc', borderless: false }}`.

Q47: What is `pressRetentionOffset`?

- **Answer:** Defines how far a user's finger can move outside the touch area before cancelling the press action.

Q48: How do you disable touch events on a `Pressable`?

- **Answer:** Pass `disabled={true}` to disable pointer interactions and ignore press callbacks.

Q49: Why might touch events feel sluggish on the JS thread, and how do modern gesture libraries solve it?

- **Answer:** Standard touch handlers process touch events on the JavaScript thread. Heavy JS execution blocks touch response. Libraries like `react-native-gesture-handler` process gestures directly on the Native UI thread.

Q50: Can `Pressable` wrap multiple children components?

- **Answer:** Yes, `Pressable` acts as a full layout container and can wrap views, text, images, and nested structures.

Section 5: The `TextInput` Component (Questions 51–65)

Q51: How do controlled vs. uncontrolled `TextInput` implementations differ in React Native?

- **Answer:**
 - **Controlled:** Driven by React state via `value` and updated via `onChangeText`.
 - **Uncontrolled:** Manages value natively via refs (`defaultValue`), avoiding state re-renders on every keypress.

Q52: What is the difference between `onChange` and `onChangeText` on `TextInput`?

- **Answer:** `onChangeText` returns the raw text string directly as an argument (`(text) => ...`). `onChange` returns the full native event object (`(e) => e.nativeEvent.text`).

Q53: What prop hides text input characters for password fields?

- **Answer:** `secureTextEntry={true}`.

Q54: How do you dismiss the software keyboard programmatically?

- **Answer:** Call `Keyboard.dismiss()` imported from `'react-native'`, or invoke `.blur()` on the `TextInput` ref.

Q55: What prop changes the action button appearance on the mobile keyboard (e.g., "Done", "Search", "Next")?

- **Answer:** `returnKeyType` (values: `'done'`, `'go'`, `'next'`, `'search'`, `'send'`).

Q56: How do you automatically focus a `TextInput` when a screen mounts?

- **Answer:** Pass `autoFocus={true}` or call `inputRef.current.focus()` inside a `useEffect` hook.

Q57: What prop specifies keyboard types (numeric, email, telephone) on `TextInput`?

- **Answer:** `keyboardType` (values: `'default'`, `'numeric'`, `'email-address'`, `'phone-pad'`, `'number-pad'`).

Q58: How do you prevent auto-correction and unwanted capitalization in user inputs?

- **Answer:** Set `autoCorrect={false}` and `autoCapitalize="none"`.

Q59: What is the `KeyboardAvoidingView` component, and why is it essential?

- **Answer:** It automatically adjusts its height, position, or bottom padding when the virtual software keyboard opens to prevent the keyboard from obscuring active input fields.

Q60: How do you handle multiline text input in React Native?

- **Answer:** Set `multiline={true}`. Optionally set `numberOfLines` (Android) or handle auto-growing height dynamically.

Q61: What prop controls the cursor / tint color inside a `TextInput`?

- **Answer:** `selectionColor`.

Q62: Why can controlled `TextInputs` suffer from "flickering" or cursor jump bugs on older devices?

- **Answer:** Asynchronous lag between the JS thread state updates and Native UI thread input updates can cause cursor repositioning loops. Uncontrolled inputs or optimized state debouncing mitigate this issue.

Q63: What prop sets placeholder text and placeholder font color?

- **Answer:** `placeholder="Enter name"` and `placeholderTextColor="#888"`.

Q64: How do you detect when the user presses the keyboard submit button?

- **Answer:** Use the `onSubmitEditing` callback prop.

Q65: What prop limits the maximum character length of an input field?

- **Answer:** `maxLength={number}`.

Section 6: `ScrollView` vs. `FlatList` & Virtualization (Questions 66–85)

Q66: What is the fundamental operational difference between `ScrollView` and `FlatList`?

- **Answer:** `ScrollView` renders **all** child nodes into memory immediately upon mounting. `FlatList` is virtualized: it lazily mounts only items visible on screen (plus a small buffer) and recycles off-screen views.

Q67: When should you use `ScrollView` instead of `FlatList`?

- **Answer:** Use `ScrollView` for small, bounded, static content layouts (like settings forms or static article pages). Use `FlatList` for long, dynamic, or unbounded data arrays.

Q68: What are the primary required props for a `FlatList`?

- **Answer:** `data` (array of items) and `renderItem` (function returning React components for each item).

Q69: Why is the `keyExtractor` prop crucial in `FlatList`?

- **Answer:** It provides a unique string key for item identity, enabling React's reconciliation engine to efficiently reorder, track, and re-render list items without full list rebuilds.

Q70: What happens if you omit `keyExtractor` and your data items lack an `id` or `key` property?

- **Answer:** React Native falls back to using the array index as the key and emits a development console warning. This can cause rendering glitches during list mutations.

Q71: How do you render list headers, footers, and empty list states in `FlatList`?

- **Answer:** Use `ListHeaderComponent`, `ListFooterComponent`, and `ListEmptyComponent`.

Q72: How do you implement pull-to-refresh on `ScrollView` or `FlatList`?

- **Answer:** Attach `refreshing={isRefreshing}` and `onRefresh={handleRefresh}` props, or pass a `<RefreshControl>` component to `refreshControl`.

Q73: What is the purpose of `getItemLayout` in `FlatList`?

- **Answer:** An optimization prop for fixed-height items. By calculating layout offsets upfront (`((data, index) => ({ length, offset, index })), FlatList` skips dynamic layout measurements, speeding up scroll performance.

Q74: What prop controls infinite scrolling / pagination loading in `FlatList`?

- **Answer:** `onEndReached` (callback when scroll position nears the bottom) combined with `onEndReachedThreshold` (defines threshold distance from bottom, e.g., `0.5`).

Q75: How does `initialNumToRender` impact application startup performance?

- **Answer:** It specifies how many items render in the initial batch. Keeping this number small speeds up initial screen mounting time.

Q76: What is `windowSize` in `FlatList`?

- **Answer:** A multiplier determining maximum viewport render area in screen-height units. Default is 21 (10 viewports above, 1 visible viewport, 10 viewports below). Lowering `windowSize` reduces memory usage at the cost of potential white flashes during fast scrolling.

Q77: Why shouldn't you nest a vertical `FlatList` inside a vertical `ScrollView` with the same orientation?

- **Answer:** It throws a *VirtualizationObserver* runtime warning because the child `FlatList` receives infinite height constraints from the parent `ScrollView`, breaking virtualization.

Q78: How do you fix nested scroll errors when combining multiple list components?

- **Answer:** Use `ListHeaderComponent` / `ListFooterComponent` inside a single `FlatList`, or set `scrollEnabled={false}` on the inner list, or use `SectionList`.

Q79: What component should be used when data needs to be categorized under sticky section headers?

- **Answer:** `SectionList` (which uses `sections` prop with `{ title, data }` structure).

Q80: What is `maxToRenderPerBatch`?

- **Answer:** Controls how many items are rendered per incremental scroll batch, balancing CPU processing time with scroll rendering speed.

Q81: How do you make header items stick to the top during scrolling in `ScrollView` or `FlatList`?

- **Answer:** Use `stickyHeaderIndices=[0]` (array of child indices that should pin to the top when scrolled past).

Q82: How do you scroll to a specific index programmatically in `FlatList`?

- **Answer:** Call `flatListRef.current.scrollToIndex({ index: targetIndex, animated: true })`.

Q83: What is the `removeClippedSubviews` prop?

- **Answer:** An optimization prop (`boolean`). When `true`, views outside the viewport are detached from the native view hierarchy to conserve memory (recommended for long lists).

Q84: How do you render a horizontal carousel using `FlatList`?

- **Answer:** Set `horizontal={true}`, `pagingEnabled={true}`, and hide indicators via `showsHorizontalScrollIndicator={false}`.

Q85: What is `extraData` prop on `FlatList`, and why is it critical?

- **Answer:** `FlatList` is a `PureComponent` that only re-renders when the data prop reference changes. If item rendering depends on external state (e.g., selection state), pass that state to `extraData` to trigger proper re-renders.

Section 7: `SafeAreaView`, `StatusBar`, & System Chrome (Questions 86–100)

Q86: What problem does `SafeAreaView` solve?

- **Answer:** It automatically applies internal padding to avoid physical hardware obstructions like display notches, camera cutouts, home indicator bars, and rounded corners on modern mobile devices.

Q87: Why is `react-native-safe-area-context` preferred over React Native's core `SafeAreaView`?

- **Answer:** The core `SafeAreaView` only supports iOS (and can have limitations on specific iOS versions). The `react-native-safe-area-context` library provides full cross-platform support (iOS & Android) with hook APIs like `useSafeAreaInsets()`.

Q88: How do you obtain pixel values for notch/home indicator dimensions programmatically?

- **Answer:** Using `useSafeAreaInsets()` from `react-native-safe-area-context`, which returns `{ top, bottom, left, right }` edge values.

Q89: What is the `StatusBar` component used for?

- **Answer:** It controls the appearance of the native device status bar (time, battery, Wi-Fi indicators at the top of the screen).

Q90: How do you change status bar text color to white for dark-themed screens?

- **Answer:** `<StatusBar barStyle="light-content"/>`.

Q91: How do you hide the status bar completely on a specific screen?

- **Answer:** `<StatusBar hidden="{true}"/>`.

Q92: How do you configure status bar background transparency on Android?

- **Answer:** Set `translucent={true}` and `backgroundColor="transparent"`.

Q93: How do you animate status bar property transitions smoothly?

- **Answer:** Use the `animated={true}` prop on `<StatusBar>`.

Q94: Why might content bleed under the status bar on Android by default?

- **Answer:** Android screens default to non-translucent status bars unless configured via `StatusBar.setTranslucent(true)` or native manifest flags.

Q95: Can you render multiple `<StatusBar>` components across different active screens in React Navigation?

- **Answer:** Yes, but the last mounted `<StatusBar>` instance overrides previous properties. Libraries like `useIsFocused()` or `FocusAwareStatusBar` components ensure proper prop synchronization per focused screen.

Q96: What happens when applying `flex: 1` directly to `SafeAreaView`?

- **Answer:** It expands to fill the entire screen while dynamically padding content inside safe device boundaries.

Q97: How do you apply safe area padding to only the top boundary (ignoring the bottom home bar)?

- **Answer:** With `react-native-safe-area-context`, set `edges={['top']}` on `<SafeAreaView>`.

Q98: Why should you avoid placing heavy background colors on `SafeAreaView` when creating custom screen headers?

- **Answer:** Applying non-matching background colors to `SafeAreaView` can create awkward colored bars at device notches. Instead, apply background colors to individual headers and apply edge insets directly.

Q99: What prop on `<StatusBar>` handles status bar style on Android dark/light themes?

- **Answer:** `barStyle` accepts `'dark-content'`, `'light-content'`, and `'default'`.

Q100: How do `SafeAreaView` and dynamic screen orientations (Landscape vs. Portrait) interact?

- **Answer:** `SafeAreaView` recalculates edge insets dynamically upon screen rotation, expanding left/right padding in landscape mode to account for side camera cutouts.